Task 1 (20 bal) “Mükəmməl ədədlər”

Riyaziyyatda özü istisna olmaqla, bütün müsbət tam bölənlərinin cəmi özünə bərabər olan ədədə **Mükəmməl ədəd** deyilir.

- Məsələn: **6** mükəmməl ədəddir. Çünki 6-nın bölənləri 1, 2 və 3-dür. Onların cəmi: 1+2+3=6.
- Məsələn: **28** mükəmməl ədəddir. Bölənləri: 1, 2, 4, 7, 14. Cəmi: 1+2+4+7+14=28.

Verilən aralıqda olan mükəmməl ədədləri ekrana çıxaran pyhton kodu yazın.

Proqramda aşağıdakı funksiyanı yazın :

- **is_perfect_number(num : int) -> bool**

Output

```
Enter start: 1
Enter end: 1000
Perfect number found: 6
Divisors: 1 2 3
Perfect number found: 28
Divisors: 1 2 4 7 14
Perfect number found: 496
Divisors: 1 2 4 8 16 31 62 124 248
Total found: 3
```

Task 2 (20 bal): "Zəncirvari Harshad" ədədləri

Riyaziyyatda rəqəmlərinin cəminə qalıqsız bölünən ədədlərə **Harshad ədədləri** deyilir. Lakin bir ədəd rəqəmləri cəminə bölündükdə alınan yeni qismət də öz rəqəmləri cəminə bölünürsə, bu, **Zəncirvari Harshad** prosesi adlanır. Proses rəqəmlər cəmi 1-ə bərabər olan ədəd alınana qədər davam edir.

Məsələn: 6804 ədədi:

1. 6+8+0+4=18. 6804/18=378 (Bölünür)
2. 3+7+8=18. 378/18=21 (Bölünür)
3. 2+1=3. 21/3=7 (Bölünür)
4. 7/7=1 (Dayanır)

Proqrama aşağıdakı funksiyaları yazın:

- **sum_digits(n: int) -> int**
  - Verilmiş ədədin rəqəmlərinin cəmini riyazi yolla hesablayıb qaytarır.
- **harshad_chain(number: int) -> int**
  - Bu funksiya ədədin zəncirvari şəkildə neçə mərhələ keçdiyini hesablayır. Hər mərhələdə ədədi, onun rəqəmlər cəmini və alınan yeni ədədi ekrana çap edir.
  - Əgər hər hansı mərhələdə ədəd rəqəmləri cəminə bölünmürsə, prosesi dayandırır və 0 qaytarır.

Output

```
Enter a number (N>0): 6804
Step 1 : 6804 / 18 = 378
Step 2 : 378 / 18 = 21
Step 3 : 21 / 3 = 7
Step 4 : 7 / 7 = 1
Total chain length: 4
```

Task 3 (20 bal): "İkili-Onluq Dominantlığı"

Bir ədəd o zaman "Dominant Səfər Ədədi" hesab olunur ki, onun onluq sistemdəki ən böyük rəqəmi, həmin ədədin ikilik (binary) qarşılığındakı "1"-lərin sayına bərabər olsun.

Proqrama aşağıdakı funksiyaları yazın:

- **max_digit(number: int) -> int**
  - Verilmiş ədədin daxilindəki ən böyük rəqəmi tapır (məs: 472 üçün 7).
- **count_binary_ones(number: int) -> int**
  - Ədədi ikilik sistemə çevirib (string istifadə etmədən), riyazi yolla daxilindəki "1"-lərin sayını hesablayır.
- **is_dominant_trip(number: int) -> bool**
  - Bu funksiya yuxarıdakı iki funksiyanı çağırır və nəticələr bərabər olduqda True, əks halda False qaytarır. Həmçinin hər iki göstəricini ekrana çap edir.

Output

```
Enter a number (N>0): 13
Max digit: 3
Binary ones count: 3
Result: True. It is a Dominant Trip number.
```

Task 4 (20 bal): "Sirkulyar-İkilik Analizator"

Bir ədəd o zaman "Sirkulyar-İkilik" (Circular-Binary) namizədi hesab olunur ki, o aşağıdakı 3 səviyyəli süzgəcdən keçsin.

Proqrama aşağıdakı funksiyaları yazın:

1. **rotate_number(n: int) -> int**
   - Ədədin ilk rəqəmini götürüb sonuna qoyaraq yeni ədəd yaradır (məs: 123→231).
2. **binary_weight_type(n: int) -> int**
   - Ədədin ikilik sistemdəki "1"-lərinin sayını tapır.
   - Əgər "1"-lərin sayı cütdürsə **2**, təkdirsə **1**, heç "1" yoxdursa (0 halı) **0** qaytarır.
3. **analyze_complex_number(n: int) -> str**
   - Bu funksiya if-elif-else bloklarından istifadə edərək ədədi aşağıdakı kateqoriyalara ayırmalıdır:
     - **Kateqoriya A:** Əgər həm ədədin özü, həm də onun fırlanmış forması (rotate_number) sadə ədəddirsə.
     - **Kateqoriya B:** Əgər ədəd sadə deyil, lakin onun binary_weight_type-ı 2-yə bərabərdirsə.
     - **Kateqoriya C:** Yuxarıdakıların heç biri deyilsə, lakin ədəd mükəmməl ədəddirsə.
     - **Default:** "Heç bir xüsusiyyət tapılmadı".

Output

```
Enter a number (N>0): 54
Original: 54
Rotated: 45
Binary Weight Type: 2
Result: Category B (Composite with Even Binary Weight)
```

Task 5 (20 bal): Faktoriyal Cəmi və Xoşbəxtlik Analizi

Istifadəçi tərəfindən n natural ədədi verilir. Proqram iki mərhələli yoxlama aparmalıdır:

1. **Birinci mərhələ:** Ədədin rəqəmlərini riyazi yolla ayıraraq hər birinin faktoriyalını (!) hesablayın və toplayın. Alınan nəticəni sum adlandıraq.
2. **İkinci mərhələ:** sum ədədinin **"Xoşbəxt ədəd" (Happy Number)** olub-olmadığını yoxlayın.

Xoşbəxt Ədəd nədir? Ədədin rəqəmlərinin kvadratlarını toplayırıq. Alınan yeni ədədlə də eyni prosesi edirik. Əgər bu prosesin sonunda **1** alınarsa, bu ədəd xoşbəxtdir. Əgər proses sonsuz dövrə girərsə (riyazi olaraq bu, **4** rəqəminə çatdıqda baş verir), ədəd xoşbəxt deyil.

Məsələn: **19** xoşbəxtdir: $1^2+9^2=82 \rightarrow 8^2+2^2=68 \rightarrow 6^2+8^2=100 \rightarrow 1^2+0^2+0^2=1$.

Aşağıdakı funksiyalardan istifadə edin :

1. **get_factorial_sum(n: int) -> int**: Bir natural ədədin rəqəmlərinin faktoriyallarını riyazi yolla toplayır.
2. **is_happy(num: int) -> bool**: Alınan bu cəmin "Xoşbəxt ədəd" olub-olmadığını yoxlayır.

Output

```
N natural ədədini daxil edin: 2221
------------------------------
Faktoriyal Cəmi (int): 7
Xoşbəxtdirmi? (bool): True
```